

تقرير التحليل المعمق لمنصة Diwan Event

بناءً على طلبك، قمت بإجراء تحليل تقني ووظيفي معمق للكود المصدري وبنية منصة "Diwan Event". بصفتي خبيراً في هندسة البرمجيات وتسيير المنصات الكبرى، أقدم لك هذا التقرير النقدي الصريح والموضوعي، والذي يسلط الضوء على الإيجابيات، السلبيات، والنقاط مقارنة بالمنصات العالمية الرائدة مثل Eventool، بالإضافة إلى مقترحات تقنية وبرومبت هندسي متكامل لتطوير المنصة.

1. الإيجابيات ونقاط القوة (The Good)

لقد تم بناء المنصة على أساس تقني صلب في بعض الجوانب، وتتضمن ميزات جيدة:

- اختيار إطار العمل (Backend): استخدام FastAPI مع Python هو اختيار ممتاز. يوفر أداءً عالياً، دعماً للبرمجة غير المتزامنة (Asynchronous)، وتوثيقاً تلقائياً للـ API (Swagger/ReDoc)، مما يجعله أساساً قوياً وقابلاً للتوسع 1.
- قاعدة البيانات: الاعتماد على PostgreSQL مع psycopg2 و ThreadedConnectionPool يضمن موثوقية عالية في إدارة البيانات والمعاملات 2.
- نظام الحضور اللحظي: توفير طرق متعددة للتسجيل (QR، USB، يدوي) مع لوحة تحكم لحظية تعتمد على WebSockets هو تطبيق عملي وفعال للفعاليات.
- توليد الشارات (Badges): استخدام reportlab لتوليد شارات PDF مع دعم ممتاز للغة العربية (RTL) باستخدام arabic_reshaper و bidi هو نقطة قوة تحسب للمنصة.
- التكامل مع بوابات الدفع: دعم Stripe للدفع الدولي و Chargily للدفع المحلي (الجزائر) يغطي احتياجات مهمة.
- النسخ الاحتياطي التلقائي: وجود نظام backup_manager.py لعمل نسخ احتياطية دورية لقاعدة البيانات وتصدير CSV للطوارئ هو ممارسة أمنية ممتازة.

2. السلبيات والنقاط (The Bad & The Missing)

بمقارنة المنصة مع المعايير العالمية ومنصات مثل Eventool، تبرز عدة نقاط معمارية ووظيفية تحتاج إلى معالجة جذرية:

أ. العيوب المعمارية والتقنية (Architectural Flaws)

- غياب (ORM) (Object-Relational Mapping): كتابة استعلامات SQL مباشرة (Hardcoded) في ملف database.py (مثل SELECT * FROM participants WHERE ...) هي ممارسة قديمة. هذا يجعل صيانة قاعدة البيانات صعبة، يعقد عملية الترحيل (Migrations)، ويزيد من احتمالية الأخطاء. المنصات الحديثة تستخدم ORM مثل SQLAlchemy 3.
- غياب نظام ترحيل قواعد البيانات (Migrations): لا يوجد أداة مثل Alembic لإدارة تغييرات مخطط قاعدة البيانات بمرور الوقت. هذا يجعل تحديث المنصة في بيئة الإنتاج محفوفاً بالمخاطر.

- **الواجهة الأمامية (Frontend) متجانسة (Monolithic):** الاعتماد على قوالب Jinja2 و Vanilla JS يجعل الواجهة الأمامية صعبة التوسع والصيانة. المنصات العالمية (مثل Eventool) تفصل الواجهة الأمامية تماماً باستخدام أطر عمل حديثة مثل React أو Vue.js، وتتواصل مع الواجهة الخلفية عبر REST APIs GraphQL 4 .
- **إدارة البيئة والإعدادات:** الاعتماد البسيط على `os.environ.get` غير كافٍ لمنصة كبرى. يجب استخدام أدوات مثل Pydantic Settings لإدارة الإعدادات والتحقق من صحتها بناءً على البيئة (Dev, Staging, Prod) 5 .
- **غياب الاختبارات الآلية (Automated Testing):** وجود ملفات اختبار بسيطة (`test_api.py`) لا يكفي. المنصات الموثوقة تتطلب تغطية اختبارات شاملة (Unit, Integration, E2E) لضمان عدم تعطل الميزات عند إضافة تحديثات جديدة.

ب. النقائص الوظيفية مقارنة بـ Eventool (Functional Gaps)

- **إدارة الفعاليات المتعددة (Multi-Event Management):** المنصة الحالية تبدو مصممة لإدارة حدث واحد في كل مرة (أو تفتقر لواجهة واضحة لإدارة أحداث متعددة بشكل مستقل). Eventool تسمح للمنظم بإدارة عشرات الفعاليات من حساب واحد، كل منها بقاعدة بيانات مشاركين وإعدادات منفصلة 6 .
- **تطبيقات الجوال الأصلية (Native Apps):** تفتقر المنصة إلى تطبيقات جوال أصلية (iOS/Android) للمشاركين. Eventool توفر تطبيقات مخصصة لكل حدث لتعزيز التفاعل وإرسال الإشعارات الفورية (Push Notifications) 7 .
- **التفاعل المباشر (Live Engagement):** ميزات التفاعل مثل الأسئلة والأجوبة (Q&A) واستطلاعات الرأي (Polls) تبدو بدائية (موجودة ككود تجريبي في `test_poll_creation.py`). المنصات العالمية توفر تجربة تفاعلية غنية ومباشرة.
- **إدارة المحتوى (CMS):** لا توجد واجهة إدارة محتوى (CMS) متكاملة تتيح للمنظمين تعديل صفحات الهبوط (Landing Pages)، جداول الأعمال، السير الذاتية للمتحدثين، ومعلومات الفنادق بسهولة دون التدخل في الكود.
- **نظام الأذونات (RBAC):** نظام الأذونات الحالي (`super_admin`, `admin`, `scanner`) بسيط جداً. المنصات الكبرى تتطلب نظام Role-Based Access Control دقيق يسمح بتخصيص صلاحيات محددة (مثلاً: مدير مالي، مدير محتوى، مسؤول تسجيل) 8 .

3. الاقتراحات التقنية للتحسين (Technical Recommendations)

للاارتقاء بمنصة Diwan Event لتنافس المنصات العالمية، أقترح تنفيذ التحديثات التالية:

1. إعادة هيكلة قاعدة البيانات (Database Refactoring):
 - الانتقال إلى استخدام SQLAlchemy كـ ORM.
 - إعداد Alembic لإدارة ترحيلات قاعدة البيانات (Migrations).
2. فصل الواجهة الأمامية (Frontend Decoupling):

****الخطوة 4: تعزيز الأمان وإدارة الإعدادات****

1. قم بتنفيذ نظام إدارة إعدادات قوي باستخدام `pydantic-settings`.
2. بدلاً من `Secret`، استخدم JWT (JSON Web Tokens) ليستخدم (Authentication) قم بتحديث نظام المصادقة.

****Frontend Architecture - React) الخطوة 5: هيكل الواجهة الأمامية****

1. للمنظمين (Next.js أو React.js) لتطبيق (Folder Structure) قدم لي هيكل مجلدات مقترح.
2. ضئها في جدول API يقوم بجلب قائمة المشاركين من الـ React (Component) اكتب نموذجاً لمكون.

****ملاحظات هامة****

- * PEP 8 ويتبع معايير (Docstrings)، اكتب كوداً نظيفاً، موثقاً.
 - * (Scalability) ركز على الأداء وقابلية التوسع.
 - * قدم الكود في أجزاء منطقية لتسهيل المراجعة والتطبيق.
1. وقدم الكود الخاص بها (SQLAlchemy نماذج) ابدأ بالخطوة

المراجع

- [1] FastAPI Documentation. (n.d.). Retrieved from
- [2] PostgreSQL Documentation. (n.d.). Retrieved from
- [3] SQLAlchemy Documentation. (n.d.). Retrieved from
- [4] React Documentation. (n.d.). Retrieved from
- [5] Pydantic Settings. (n.d.). Retrieved from
- [6] Eventool Features. (n.d.). Retrieved from
- [7] Expo Documentation. (n.d.). Retrieved from
- [8] Role-Based Access Control (RBAC). (n.d.). Retrieved from